Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Правила разработки с ИИ-ассистентами

Процесс, ограничения и ревью · версия 0.6.0

29 октября 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Принципы 4

3 Соглашения по коду 5

4 Шаблон задачи для ассистента 6

5 Ветки, коммиты, слияние 7

6 Готовность задачи 8

7 Обязательное ревью человека 9

8 Что запрещено ассистенту 10

9 Работа с данными 11

10 Цикл истории 12

1 О документе

Документ устанавливает правила работы ИИ-ассистентов (Claude Code, ZCode и подобных) над репозиторием «Планировщика отпусков» и объясняет, почему правила именно такие. Цель — предсказуемый результат: ассистент закрывает историю бэклога с проверяемыми критериями, человек отвечает за решения.

Рабочая инструкция для самого ассистента — `AGENTS.md` в корне репозитория: структура, команды, жёсткие правила, порядок выполнения задачи. Этот файл — операционная истина; документ 6 — его обоснование и процесс вокруг него.

СВЯЗАННЫЙ ДОКУМЕНТ	ЧТО В НЁМ
3 — План развития и бэклог	Истории с критериями приёмки
7 — Стенды, CI/CD и выпуск	Ветки, конвейеры, порядок выпуска
10 — Промпты итераций	Готовые формулировки задач

2 Принципы

1. **История — это задача.** Ассистент получает одну историю бэклога целиком: ID, критерии, ссылки на документы и операции контракта. Без истории — только исследование и вопросы.
2. **Тесты до кода.** Сначала падающие тесты по критериям приёмки, затем минимальная реализация, затем зелёный прогон.
3. **Контракт и спецификации меняет человек.** `contracts/` и `docs/specs/` правятся только через ревью владельца (CODEOWNERS). Если ассистенту «не хватает» поля в API, он останавливается и спрашивает.
4. **Человек решает, ассистент исполняет.** Архитектура, безопасность, права и миграции данных — обязательное ревью технического лидера.
5. **Ничего лишнего.** Никаких зависимостей, функций и рефакторинга за пределами истории.
6. **Нестабильный тест — это дефект.** Тесты не отключаются, не пропускаются и не ослабляются; ищется причина.

3 Соглашения по коду

ОБЛАСТЬ	ПРАВИЛО
Сервер	NestJS 11, TypeScript strict; домен — `server/src/<домен>/` : тонкий контроллер (DTO-валидация → сервис → problem+json) и сервис с логикой
Чистые вычисления	Баланс, интервалы, аналитика — модули без БД (`balance.ts` , `coverage.ts`); дата расчёта передаётся аргументом
Время	`new Date()` допустим только для «сейчас» в точке входа сервиса
Транзакции	Изменение данных, аудит и уведомления — в одном `prisma.$transaction` ; гонки закрываются `SELECT … FOR UPDATE`
Ошибки	Только problem+json с кодом из каталога документа 2; новые коды не придумываются
Пароли и секреты	Только `auth/passwords.ts` (scrypt); секреты из окружения; в журналы, ответы и выгрузки не попадают
Клиент	Vue 3 + Pinia без компонентных библиотек; `web/src/api/client.ts` — единственный модуль с `fetch` ; токен в памяти
Вёрстка	Цвета и отступы — токены `tokens.css` ; пользовательские данные — интерполяцией Vue; `v-html` запрещён
Тесты	Сервер: `server/test/*.unit.test.ts` , `*.e2e.test.ts` на тестовой БД `vacation_test` ; клиент: `web/src/lib/*.spec.ts` и Playwright `web/e2e`

4 Шаблон задачи для ассистента

```
## Задача {ID}. {Название}
Ссылки: docs/specs/03-backlog.md ({эпик}), docs/specs/02-api-spec.md ({раздел}),
contracts/openapi-vacation-api-v1.yaml (operationId {…}).
Контекст: {что уже есть; какие файлы; где смотреть}
Критерии приёмки:
- {критерий из бэклога}
Ограничения: {что не трогать; миграции; зависимости}
На выходе: код + тесты + CHANGELOG; ревью ТЛ: {да/нет}.
```

5 Ветки, коммиты, слияние

- Работа над версией — в ветке `v0.x.0`; история — в ветке `story/<id>` от ветки версии.
- Сообщение коммита: тип и область, затем суть по-русски: `fix(v0.3.0): гонки заявок и «память» переноса`.
- Одна история — один PR по шаблону: что сделано, как проверено, решения, что осталось.
- Слияние только при зелёных конвейерах GitHub и Gitea.

6 Готовность задачи

Ассистент сообщает о завершении только после того, как:

1. `make lint`, `make test`, `cd web && npm test` и сборка обоих пакетов зелёные; при изменении экранов — `npm run e2e`;
2. критерии приёмки отмечены со ссылками на тесты;
3. при изменении API — обновлён контракт, тест сверки контракта зелёный;
4. при новой зависимости — запись в `docs/decisions.md` и зелёный `make sbom`;
5. `CHANGELOG.md` обновлён, а «что осталось» перечислено честно.

7 Обязательное ревью человека

`server/src/auth/`, `server/prisma/migrations/`, `contracts/`, `docs/specs/`, `.github/workflows/`, `.gitea/workflows/`, матрица прав, `scripts/license-report.mjs`, любые зависимости. Сигнал остановиться и спросить — выбор между вариантами поведения, которые не описаны в документах.

8 Что запрещено ассистенту

1. Менять контракт и спецификации без ревью владельца.
2. Добавлять зависимости без обоснования и SBOM-отчёта.
3. Трогать рабочую БД: тесты используют `vacation_test`.
4. Оставлять пароли, токены и секреты в коде, тестах, журналах и выгрузках.
5. «Улучшать» то, что не входит в историю.
6. Отключать, пропускать или ослаблять тесты, чтобы получить зелёный CI.
7. Отмечать историю выполненной без прохождения проверок.

9 Работа с данными

Ассистенту доступны демо-данные `server/prisma/seed-data` и синтетические фикстуры тестов. Рабочая БД и реальные персональные данные ассистенту не передаются; для новых тестовых данных используются вымышленные ФИО.

10 Цикл истории

План истории (P01) → тесты, красные (P02) → реализация (P03) → самопроверка (P04) → исправление CI (P05) → независимое ревью вторым ассистентом (P06) → критическое ревью человека для историй с ревью ТЛ (P07) → документация (P08) → закрытие истории (P09). Разбор дефекта — P11, остановка и эскалация — P12, закрытие итерации — P10. Тексты промптов — в документе 10 и каталоге `prompts/base/`.